Unit II Linear Data Structures Using Sequential Organization

Sequential Organization, Linear Data Structure Using Sequential Organization, Array as an Abstract Data Type, Memory Representation and Address Calculation, Inserting an element into an array, Deleting an element, Multidimensional Arrays, Two-dimensional arrays, n-dimensional arrays, Concept of Ordered List, Single Variable Polynomial, Representation using arrays, Polynomial as array of structure, Polynomial addition, Polynomial multiplication, Sparse Matrix, Sparse matrix representation, Sparse matrix addition, Transpose of sparse matrix.

Case Study- Use of sparse matrix in Social Networks and Maps

Sequential Organization:-

- Organization of data within the computer memory is linear i.e. each element has unique successor.
- Direct or Random access to any element is the major advantage of sequential organization.
- Sequential organization makes it possible to access any element in the data structure randomly in constant time O (1).
- The two most natural ways of storing sequences in computer memory are arrays and linked lists. We model the memory as a sequence of memory cells, each of which has a unique address (a 32 bit non-negative integer on a 32-bitmachine). An array is simply a contiguous piece of memory, each cell of which stores one object of the sequence stored in the array (or rather a reference to the object). In a singly linked list, we allocate two successive memory cells for each object of the sequence. These two memory cells form a node of a sequence .The first stores the object and the second stores a reference to the next node of the list (i.e., the address of the first memory cell of the next node)
- The first step in designing a data structure is to develop a mathematical model for the data to be stored. Then we decide which methods we need to access and modify the data. Such a model, together with the methods to access and modify it, is an abstract data type (ADT). An ADT completely determines the functionality of a data structure (what we want it to), but it says nothing about the implementation of the data structure and its methods (how the data structure is organized in memory, or which algorithms we implement for the methods).Clearly we will be very interested in which algorithms are used but not at the stage of defining the ADT. The particular algorithms/data structures that get used will influence the running time of the methods of the ADT. The definition of an ADT is something done at the beginning of a project, when we are concerned with the specification1of a system. As an example, if we are implementing a dictionary ADT, we will need to perform operations such as look-up (w), where w is a word. We would know that this is an essential operation at the specification stage, before deciding on data structures or algorithms.

- Sequential Organization Example : Arrays
- Array is an ordered set of objects.
- Data structure that stores data of similar type at consecutive memory locations.
- e.g. int A[5]; // A is an array of size 5
- It will allocate 5 consecutive memory locations for the array A. Each element of array can be accessed by referring to its index.
- The term “linear” means “belonging to a line”
- A Linear data structure consists of elements which are ordered i.e. in a line.
- The elements form a sequence such that there is a first element, second, ... and last element.
- Thus, the elements of a linear data structure have a one-one relationship.
- Examples: array, list.

Array:-

- An array is a finite ordered set of homogenous elements.
- **Finite :** There are a specific number of elements in the array.
- **Ordered :** The elements are arranged so that there is a first element, second, and so on.
- **Homogenous :** All the elements are of the same type.
- Mathematically, an array can be considered as a set of pairs- (index, value). For each index, there is an associated value. This means that, there is a one-to-one mapping or correspondence between the set of indices and the set of values.
- A single dimension array is created as :
- <data type> <array name> [size] ;
- Example
- int A[5];
- The array is implemented as a constant pointer to the array elements.
- The elements are stored in consecutive memory locations. The address of the first element is called the base address.

- **Primitive Operations on Arrays:-**

- **Create:** Creating a new array: This needs the information regarding the type of data objects and the number of data objects to be stored.

- **Store (array, index, value) :** Store a value at particular position : This needs the name of the array, the position in the array and the value to be stored at that position.

- **Retrieve (array, index):** Retrieve a value at a particular position: This operation requires the array name and an index and it returns the corresponding value.

• Array as an ADT :-

Objects: A set of pairs <index, value> where for each value of index there is a value from the set item. Index is a finite ordered set of one or more dimensions, for example, {0, … , n-1} for one dimension,

{(0,0),(0,1),(0,2),(1,0),(1,1),(1,2),(2,0),(2,1),(2,2)} for two dimensions, etc.

Methods:

for all A ∈ Array, i ∈ index, x ∈ item, j, size ∈ integer

Array Create(j, list) ::= **return** an array of j dimensions where list is a j-tuple whose kth element is the size of the kth dimension. Items are undefined.

Item Retrieve (A, i) := **if** (i ∈ index) **return** the item associated with index value i in array A
else return error

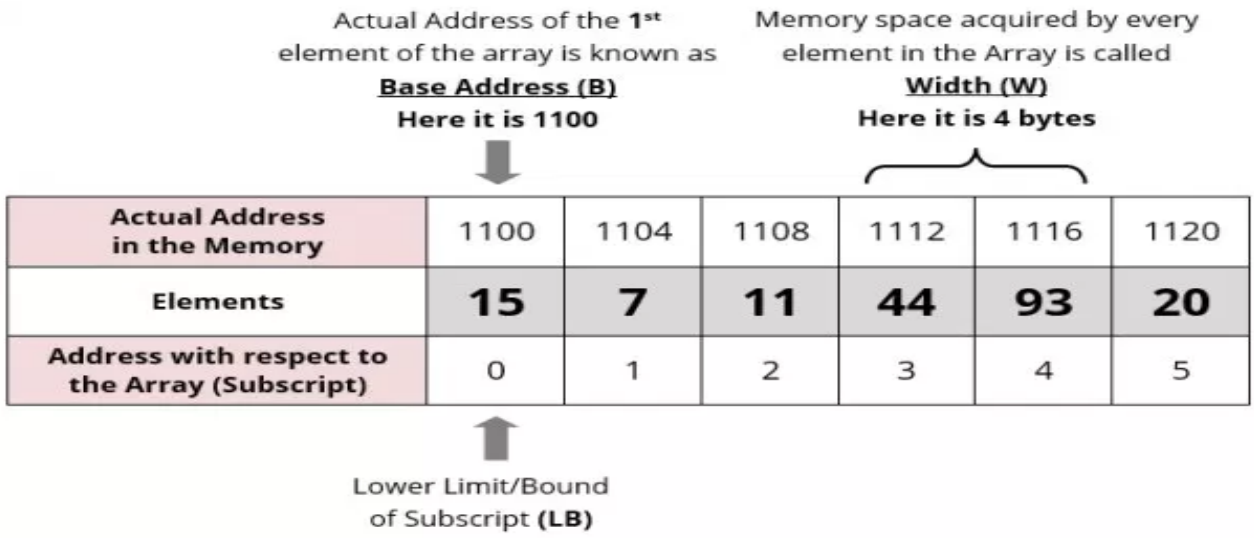
Array Store (A, i, x) := **if (**i in index)
return an array that is identical to array A except the new pair <i, x> has been inserted **else return** error

Memory Representation:-

0	1	2	3	4
10	20	30	40	50
2000	2004	2008	2012	2016
A[0]	A[1]	A[2]	A[3]	A[4]

- The lower bound of array starts at 0 and upper bound is (size-1)
- The operation store is implemented as :
  - arrayname[position] = value ;
  - e.g. A[2] = 30;
- The operation extract is implemented as :
  - arrayname[position]
  - e.g. num = A[2];
- Any reference to element A[i] is converted to a pointer notation as
  - A[i] = i[A] = *(A + i)

Address Calculation in single (one) Dimension Array:



Address of A [I] = B + W * (I – LB)

Where,
B=Base address

W=Storage Size of one element stored in the array (in byte)
I = Subscript of element whose address is to be found

LB = Lower limit / Lower Bound of subscript, if not specified assume 0 (zero)

Example:

Given the base address of an array **B[1300…..1900]** as 1020 and size of each element is 2 bytes in the memory. Find the address of **B [1700]**.

Solution:

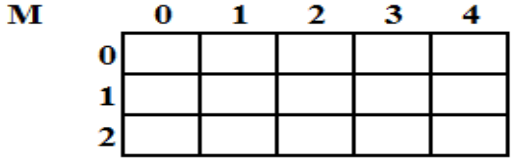
The given values are: B = 1020, LB = 1300, W = 2, I = 1700

Address of A [I] = B + W * (I – LB)

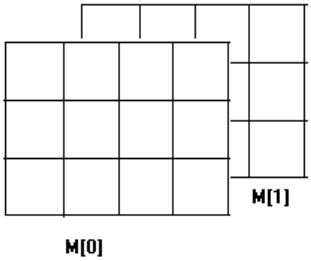
= 1020 + 2 * (1700 – 1300)
= 1020 + 2 * 400
= 1020 + 800
= 1820

Multidimensional Array: =

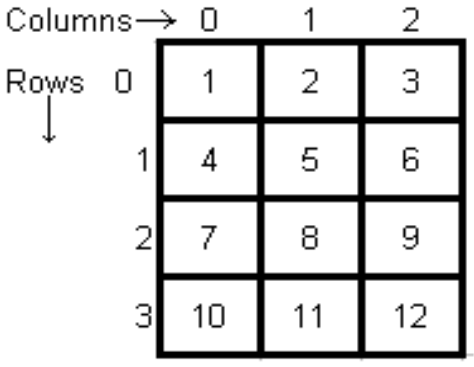
- When each array element is another array, it is called a multidimensional array.
- Example:
  - int M[3][5];
  - Declares M as an array containing 3 elements each of which is an array containing 5 elements.
  - An element of this array is accessed by specifying two indices. Such an array is called a two-dimensional array.
- M is said to have 3 rows and 5 columns. The rows are numbered 0 to 2 and columns are numbered 0 to 4.



- An array can have more than two dimensions. For example, a three dimensional array may be declared as :
  - int M[2][3][4];
  - M is an array containing 2 two-dimensional arrays, each having 3 rows and 4 columns.

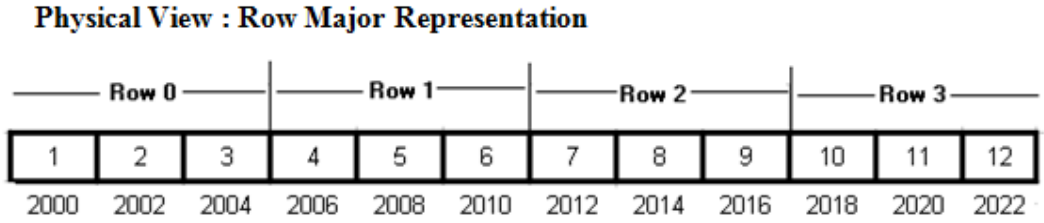


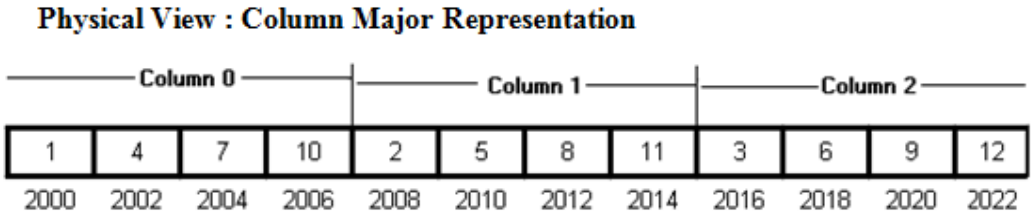
- A two-dimensional array is a logical data structure that is useful in describing an object that is physically two-dimensional such as a matrix or a check board. This is the logical view of data.
- However, computer memory is linear, i.e. it is essentially a one-dimensional array. Thus, the elements of a two-dimensional array have to be stored linearly in memory.
- They can be represented in two ways.
  - **Row-major representation:** In this representation, the elements are arranged row-wise i.e. the 0th row elements are stored first, the next row elements after them and so on.
  - **Column-major representation :** Elements are arranged column-wise i.e. all elements of the 0th column occupy the first set of memory locations, elements of the first column come next and so on.



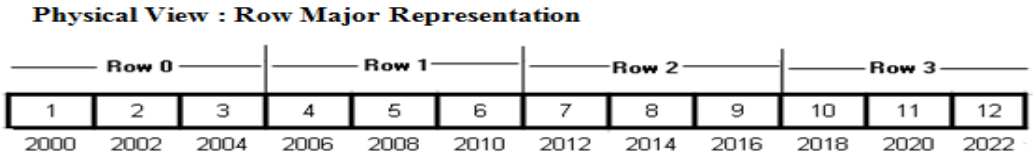
Logical View

Physical View:-





Address Calculation:-



For an array int M[R][C]; where R = number of rows and C = number of columns , the location of an element M[i][j] in the array can be calculated as :

Address of M[i][j] = Base Address + i * C * Element size + j * Element size

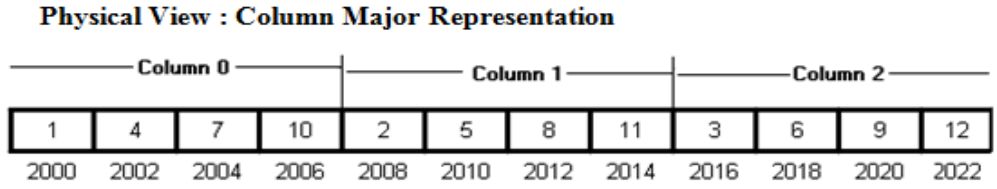
= Base Address + (i * C + j) * Element size

Example:

Address of M[1][2] = 2000 + (1 * 3 + 2) * sizeof (int)

= 2000 + 5 * 2

= 2010



For an array int M[R][C]; where R = number of rows and C = number of columns , the location of an element M[i][j] in the array can be calculated as :

Address of M[i][j] = Base Address + j * R * Element size + i * Element size

= Base Address + (j * R + i) * Element size

Example :

Address of M[1][2] = 2000 + (2 * 4 + 1) * sizeof (int)

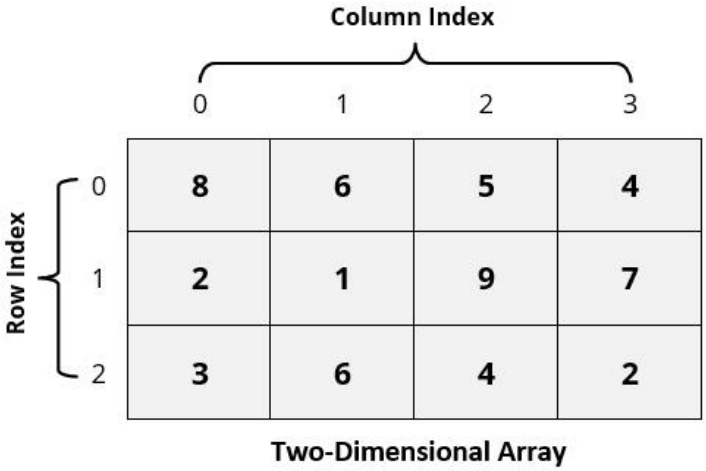
= 2000 + 9 * 2

= 2018

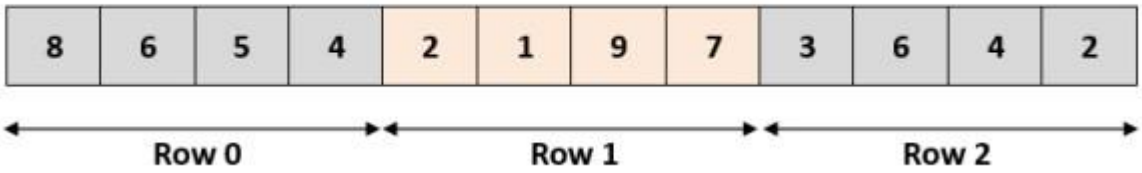
Address Calculation in Two Dimensional Array:2-D array:-

(Note: - This is another way to calculate the address whichever you feel easy do that way)

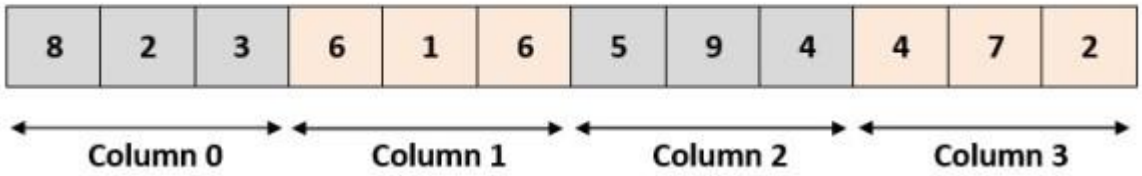
While storing the elements of a 2-D array in memory, these are allocated contiguous memory locations. Therefore, a 2-D array must be linearized so as to enable their storage. There are two alternatives to achieve linearization: Row-Major and Column-Major.



Row-Major (Row Wise Arrangement)



Column-Major (Column Wise Arrangement)



Address of an element of any array say "**A[I][J]**" is calculated in two forms as given:
(1) Row Major System (2) Column Major System

Row Major System:

The address of a location in Row Major System is calculated using the following formula:

Address of A [I][J] = B + W * [N * (I – Lr) + (J – Lc)]

Column Major System:

The address of a location in Column Major System is calculated using the following formula:

Address of A [I] [J] Column Major Wise = B + W * [(I – Lr) + M * (J – Lc)]

Where,
B = Base address
I = Row subscript of element whose address is to be found
J = Column subscript of element whose address is to be found
W = Storage Size of one element stored in the array (in byte)
Lr = Lower limit of row/start row index of matrix, if not given assume 0 (zero)
Lc = Lower limit of column/start column index of matrix, if not given assume 0 (zero)
M = Number of row of the given matrix
N = Number of column of the given matrix

Usually number of rows and columns of a matrix are given (like A[20][30] or A[40][60]) but if it is given as **A[Lr- – – – – Ur, Lc- – – – – Uc]**. In this case number of rows and columns are calculated using the following methods:

Number of rows (**M**) will be calculated as = **(Ur – Lr) + 1**

Number of columns (**N**) will be calculated as = **(Uc – Lc) + 1**

And rest of the process will remain same as per requirement (Row Major Wise or Column Major Wise).

Examples:

Q 1. An array X [-15…….10, 15……40] requires one byte of storage. If beginning location is 1500 determine the location of X [15][20].

Solution:

As you see here the number of rows and columns are not given in the question. So they are calculated as:

Number or rows say **M = (Ur – Lr) + 1** = [10 – (- 15)] +1 = 26
Number or columns say **N = (Uc – Lc) + 1** = [40 – 15)] +1 = 26

(i) Column Major Wise Calculation of above equation

The given values are: B = 1500, W = 1 byte, I = 15, J = 20, Lr = -15, Lc = 15, M = 26

Address of A [I][J] = B + W * [(I – Lr) + M * (J – Lc)]

= 1500 + 1 * [(15 – (-15)) + 26 * (20 – 15)] = 1500 + 1 * [30 + 26 * 5] = 1500 + 1 * [160]

= 1660

(ii) Row Major Wise Calculation of above equation

The given values are: B = 1500, W = 1 byte, I = 15, J = 20, Lr = -15, Lc = 15, N = 26

Address of A [I][J] = B + W * [N * (I – Lr) + (J – Lc)]

= 1500 + 1* [26 * (15 – (-15))) + (20 – 15)] = 1500 + 1 * [26 * 30 + 5] = 1500 + 1 * [780 + 5] = 1500 + 785

= 2285

Concept of Ordered List:-

- Ordered List is nothing but a set of elements. Such a list sometimes called as linear list.
- **Definition:** An ordered list is set of elements where set may be empty or it can be written as a collection of elements such as (a1,a2,a3,……,an) .
- Example :

–Days of the week

{Mon, Tue, Wed, Thu, Fri, Sat, Sun }

–Months of the year

{ Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec}

–Values on the card deck

{2, 3, 4, 5, 6, 7, 8, 9, 10, J, Q, K, A}

–List of one digit number

{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

- **Operations on ordered list**
    1. Finding the length of the list.
    2. Retrieving the ith element from the list.
    3. Display of list
    4. Searching a particular element from the list.
    5. Insertion of any element in the list.
    6. Deletion of any element from the list.
- **Applications of Ordered List**
    1. Polynomial Representation
    2. Stacks
    3. Queues, etc.

Polynomial representation using array:-

- **What is Polynomial?**
- A polynomial is an expression that contains more than two terms. A term is made up of coefficient and exponent.
  Example: $P(x) = 4x^3+6x^2+7x+9$
- Array representation assumes that the exponents of the given expression are arranged from 0 to the highest value (degree), which is represented by the subscript of the array beginning with 0. The coefficients of the respective exponent are placed at an appropriate index in the array. The array representation for the below polynomial expression is given below:
  $P(x) = x^2 - 4x + 7$.
- A polynomial may be represented using array or structure. A structure may be defined such that it contains two parts – one is the coefficient and second is the corresponding exponent. The structure definition may be given as shown below:

```
Struct polynomial{
    int coefficient;
    int exponent;
};
```

The basic idea of polynomial addition is to add coefficient parts of the polynomials having same exponent.

Polynomials

$A(X) = 3X^{20}+2X^5+4$

$B(X) = X^4+10X^3+3X^2+1$

Abstract data type *Polynomial*

Structure *Polynomial* is
objects: a set of ordered pairs of $<e_i,a_i>$ where a_i in *Coefficients* and e_i in *Exponents*, e_i are integers >= 0
functions:
for all *poly, poly1, poly2* Î *Polynomial*, *coef* Î*Coefficients*, *expon* Î*Exponents*

Polynomial Zero( ) ::= **return** the polynomial, $p(x) = 0$

Boolean IsZero(*poly*) ::= **if** (*poly*) **return** *FALSE*

else return *TRUE*

Coefficient Coef(*poly, expon*) ::= **if** (*expon Î poly*) **return** its coefficient **else return** Zero

Exponent Lead_Exp(*poly*) ::= **return** the largest exponent in *poly*

Polynomial Attach(*poly,coef, expon*) ::= **if** (*expon Î poly*) **return** error **else return** the polynomial poly with the term <*coef, expon*> inserted

Polynomial Remove(*poly, expon*) ::= **if** (*expon Î poly*) **return** the polynomial *poly* with the term whose exponent is e*xpon deleted* **else return** error

Polynomial SingleMult(*poly, coef, expon*) ::= **return** the polynomial *poly* • *coef* • x^{expon}

Polynomial Add(*poly1, poly2*) ::= **return** the polynomial *poly*1 +*poly*2

Polynomial Mult(*poly1, poly2*) ::= **return** the polynomial *poly*1 • *poly*2

End *Polynomial*

Algorithm (Polynomial Addition):-

AddPoly(Struct Poly p1[10],Struct Poly p2[10],int t1,int t2,Struct Poly p3[10])

[Initialize segment variables]

[Initialize Counter] Set i=0,j=0,k=0

2.) Repeat step 3 while i<t1 and j<t2

3.) If p1[i].expo=p2[j].expo, then

p3[i].coeff=p1[i].coeff+p2[i].coeff

p3[k].expo=p1[i].expo

[Increase counter] Set i=i+1,j=j+1,k=k+1

else if p1[i].expo > p2[j].expo, then

p3[k].coeff=p1[i].coeff

p3[k].expo=p1[i].expo

[Increase counter] Set i=i+1,k=k+1

else

p3[k].coeff=p2[j].coeff

p3[k].expo=p2[j].expo

Set j=j+1,k=k+1

[End of If]

[End of loop]

4.) Repeat while i<t1

p3[k].coeff=p1[i].coeff

p3[k].expo=p1[i].expo

Set i=i+1,k=k+1

[End of loop]

5.) Repeat while j<t2

p3[k].coeff=p2[j].coeff

p3[k].expo=p2[j].expo

Set j=j+1,k=k+1

[End of loop]

6.) Return k

7.) Exit

Other approach to represent polynomial:-

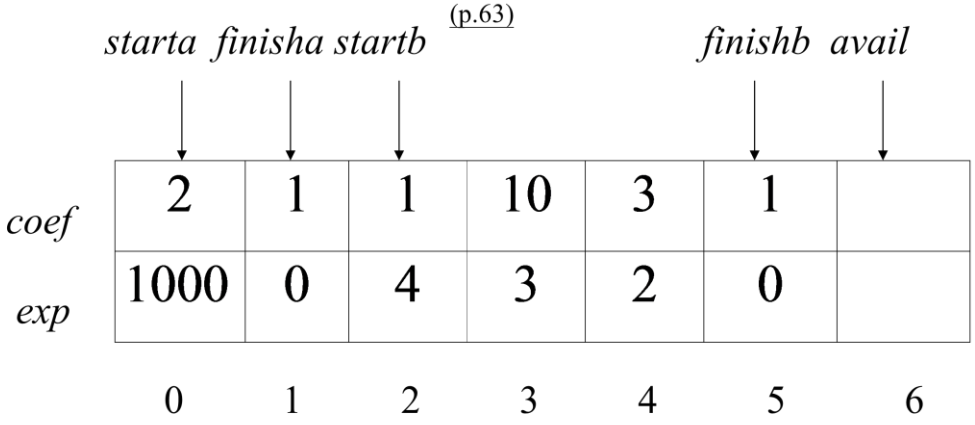
Use one global array to store all polynomials

Data structure 2: use one global array to store all polynomials

$A(X)=2X^{1000}+1$

$B(X)=X^4+10X^3+3X^2+1$

***Figure 2.2:** Array representation of two polynomials (p.63)



specification	representation
poly	<start, finish>
A	<0,1>
B	<2,5>

Multiply two polynomials:-

- Given two polynomials represented by two arrays, write a function that multiplies given two polynomials.
- Example:
- Input:

  A[] = {5, 0, 10, 6}

  B[] = {1, 2, 4}

  Output: prod [] = {5, 10, 30, 26, 52, 24}

  The first input array represents "5 + 0x^1 + 10x^2 + 6x^3"

  The second array represents "1 + 2x^1 + 4x^2"

  And Output is "5 + 10x^1 + 30x^2 + 26x^3 + 52x^4 + 24x^5"

- A simple solution is to one by one consider every term of first polynomial and multiply it with every term of second polynomial. Following is algorithm of this simple method.
- **multiply (A[0..m-1], B[0..n01])**

  1) Create a product array prod[] of size m+n-1.

  2) Initialize all entries in prod[] as 0.

  3) Traverse array A[] and do following for every element A[i] ...

  (3.a) Traverse array B[] and do following for every element B[j]

  prod[i+j] = prod[i+j] + A[i] * B[j]

  4) Return prod[].

Sparse Matrix:-

- Matrices play a very important role for solving many problems in scientific applications.
- Therefore, they have to be efficiently represented such that the operations on matrices like addition, multiplication, inverse, transpose etc. can be carried out with minimum time and space requirements.
- **Definition of Sparse Matrix:** A Sparse matrix is a matrix having very few non-zero elements and a large number of elements are Zero.
- Example :

$$M = \begin{bmatrix} 10 & 0 & 0 & 0 & -5 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -4 & 0 & 0 & 0 \\ 7 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 9 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Efficient representation of Sparse Matrix:-

- A Sparse matrix can be stored as a list of tuples storing only the non-zero elements and their row and column number.

- The first tuple gives the information about the matrix namely number of rows, number of columns and the number of non-zero elements. The remaining gives information about each non-zero value.
- Data Structure
  - 2-D Array
  - Array of Structure

Row No	Column No	Value
5	**8**	**6**
0	0	10
0	4	-5
1	2	1
2	4	-4
3	0	7
4	1	9

SPARSE MATRIX ABSTRACT DATA TYPE:-

Structure *Sparse_Matrix* is
objects: a set of triples, <*row, column, value*>, where *row* and *column* are integers and form a unique combination, and *value* comes from the set *item.*
functions:
for all *a, b* ∈ *Sparse_Matrix*, *x* Î *item, i, j, max_col, max_row* Î *index*

Sparse_Marix Create(*max_row, max_col*) ::=
return a *Sparse_matrix* that can hold up to *max_items* = *max _row ´ max_col* and whose maximum row size is *max_row* and whose maximum column size is *max_col.*

Sparse_Matrix Transpose(*a*) ::=
return the matrix produced by interchanging the row and column value of every triple.

Sparse_Matrix Add(*a, b*) ::=
if the dimensions of a and b are the same

return the matrix produced by adding corresponding items, namely those with identical *row* and *column* values.
else return error

Sparse_Matrix Multiply(*a, b*) ::=
if number of columns in a equals number of rows in **b**
return the matrix *d* produced by multiplying a by *b* according to the formula: *d* [*i*] [*j*] = S(a[i][k]•b[k][j]) where *d (i, j)* is the *(i,j)*th element
else return error.

Addition of sparse matrix:-

Sparse Matrix A

Row No	Column No	Value
5	**8**	**6**
0	0	10
0	4	-5
1	2	1
2	4	-4
3	0	7
4	1	9

+

Sparse Matrix B

Row No	Column No	Value
5	**8**	**5**
0	1	2
0	4	7
1	1	3
1	2	2
3	0	4

Addition Result

Row No	Column No	Value
5	**8**	**8**
0	0	10
0	1	2
0	4	2
1	1	3
1	2	3
2	4	-4
3	0	11
4	1	9

Transpose of a sparse matrix:-

Sparse Matrix A

Row No	Column No	Value
5	8	6
0	0	10
0	4	-5
1	2	1
2	4	-4
3	0	7
4	1	9

Transpose

Row No	Column No	Value
8	5	6
0	0	10
0	3	7
1	4	9
2	1	1
4	0	-5
4	2	-4

Simple Transpose of a Sparse Matrix:-

- If we simply interchange the row and column numbers, the elements will not be in the proper order i.e. the element A (row,col,value) will be at T(col, row,value) but we do not know where to put the element.
- If we store them consecutively, we will have to move tuples every time a new tuple has to be inserted to maintain the sequence.
-
- This can be avoided by finding all elements in column i of A and put them in row i of T
- for all $0 \leq i < num - cols$.
- i.e. for all columns j in matrix A do
- for all elements in matrix A do
- place A(i , j, value) in T(j,i, value)

```
void Simple_transpose_sparse_matrix(struct sparse s[],struct sparse t[])
{     int i,j,k=1;
      for(i=0;i<s[0].col;i++)
     {
          for(j=1;j<=s[0].val;j++)
          {
               if(s[j].col == i)
              {
                    t[k].row = s[j].col;
                  t[k].col = s[j].row;
                    t[k].val = s[j].val;
                    k++;
              }
         }
    }
    t[0].row = s[0].col;
    t[0].col = s[0].row;
    t[0].val = s[0].val;
}
```

Analysis of Simple Transpose:-

- If there are n columns in matrix A of order m x n and the number of non-zero terms is t, the computing time is O(nt).
- If t is the order of nm, the time for this algorithm becomes $O(n^2m)$ which is far worse than O(nm) time for transpose of normal matrices.

Fast Transpose of a Sparse Matrix:-

- This method achieves the transpose of a sparse matrix in O(n+t) time and hence is much faster than the simple transpose method. This method is as follows
  - Find the number of elements in each column of A
  - This gives the number of elements in each row of T
  - Using this, calculate the starting point of each row of T.
  - Move each element of A one by one to its correct position in T.
  - Example:-

Row No	Column No	Value
5	8	6
0	0	10
0	4	-5
1	2	1
2	4	-4
3	0	7
4	1	9

Number of elements in each column of A

	0	1	2	3	4	5	6	7
Count	2	1	1	0	2	0	0	0

The starting positions of each row in T.

	0	1	2	3	4	5	6	7
Position	0	2	3	4	4	6	6	6

Row No	Column No	Value
5	8	6
0	0	10
0	4	-5
1	2	1
2	4	-4
3	0	7
4	1	9

Row No	Column No	Value
5	8	6
0	0	10
0	3	7
1	4	9
2	1	1
4	0	-5
4	2	-4

```
void Fast_transpose_sparse_matrix(struct sparse s[],struct sparse t[])
{
   static int pos[15],term[15];
   t[0].row = s[0].col;
   t[0].col = s[0].row;
```

```
    t[0].val = s[0].val;
    for(i=1;i<=s[0].val;i++)
          term[s[i].col]++;
    pos[0] = 1;
    for(i=1;i<s[0].col;i++)
       pos[i] = pos[i-1] + term[i-1];
    for(i=1;i<=s[0].val;i++)
    {
        t[pos[s[i].col]].row = s[i].col;
        t[pos[s[i].col]].col = s[i].row;
        t[pos[s[i].col]].val = s[i].val;
        pos[s[i].col]++;
    }
}
```

Analysis of Fast Transpose

- The steps taken to find number of elements in each column of A = n [Assuming a m x n matrix]
- The steps taken to compute the starting positions of each row in T = n -1
- The elements of A are copied to T in t steps
- Therefore, Total no. of steps = n + n – 1 + t
- Which is of the order O(n+t).
- If t is of the order nm, this method becomes O(nm) which is same for two dimensional arrays.
- Application of Sparse Matrix
    - Social Networks
    - Maps